2. EfficientNet Model

Using `import tensorflow as tf` to import TensorFlow so that I can use it's library, while the command `from tensorflow.keras import layers, models` brings in Keras tools from TensorFlow. The command `imports matplotlib.pyplot as plt` imports Matplotlib's plotting module and finally `import numpy` as np imports NumPy. Then to connect the drive to the document I use the command `DATASET_DIR = '/content/drive/MyDrive/Colab Notebooks/Software Technology 1 G (8995)/Week 7/Assets/insects_dataset/train_data/'` to link them together.

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
import os
```

```
DATASET_DIR = '/content/drive/MyDrive/Colab Notebooks/Software Technology 1 G (8995)/Week 7/Assets/insects_dataset/train_dat
```

```
IMG_SIZE = (224, 224)        # EfficientNetB0 default input size
BATCH_SIZE = 32
SEED = 123
```

Task2_1

Using this code, I can load the images from the `train_data` folder and store them in `DATASET_DIR` which automatically splits them into two parts: 80% for training `(train_ds)` and 20% for validation `(val_ds)`. After this the function `tf.keras.preprocessing.image_dataset_from_directory()` will read the images from the subfolders where it will resize all the images and name them `IMG_SIZE` and group them into batches of size `BATCH_SIZE` and using the `SEED` value to split them. After this the `train_ds.class_names` will store the names, `len(class_names)` will count how many classes there are and `print("Classes:", class_names)` displays the output.

```
# === STEP 1: Load dataset with 80/20 train-validation split ===

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

class_names = train_ds.class_names
num_classes = len(class_names)
print("Classes:", class_names)
```

```
Found 27 files belonging to 3 classes.
Using 22 files for training.
Found 27 files belonging to 3 classes.
Using 5 files for validation.
Classes: ['Leptophlebiidae sp', 'Leuctra sp', 'Limnius sp']
```

Task 2_2

Using this code will help improve the overall performance of the datasets. `AUTOTUNE = tf.data.AUTOTUNE` tells TensorFlow to choose the best resources required for the upcoming tasks so that the user does not need to set them manually. `train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)` and `val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)` apply the best optimisation for the dataset.

```
# === STEP 2: Optimize dataset performance ===

AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
```

Task 2_3

The code uses `tf.keras.Sequential([...])` to build a sequence for the selected images transformation while `layers.RandomFlip("horizontal")` rotated those images from left to right. `layers.RandomRotation(0.1)` slightly rotates them and `layers.RandomZoom(0.1)` zooms in or out accordingly. The second part of the code is to display the images in a 3 by 3 grid.

```
# === STEP 3: Data augmentation ===

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

# Optional: visualize augmented images
plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    augmented = data_augmentation(images)
    for i in range(9):
        ax = plt.subplot(3, 3, i+1)
        plt.imshow(augmented[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
plt.show()
```

Task 2_4

This code builds transfer learning models with `EfficientNetB0`. Using `include_top=False` removes EfficientNet's original final classification layer so that I can use my own classes and then take input images with shape IMG_SIZE + (3,) which allows for data augmentation and preprocesses the images into the format EfficientNet expects. Using `GlobalAveragePooling2D()` to make a simpler form, Dropout(0.3) reduces overfitting by 30%, `Dense(num_classes, activation='softmax')` creates the final output layer and then `model.summary()` displays an overview.

```
# === STEP 4: Build transfer learning model with EfficientNetB0 ===

base_model = tf.keras.applications.EfficientNetB0(
    input_shape=IMG_SIZE + (3,),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False  # freeze base

inputs = layers.Input(shape=IMG_SIZE + (3,))
x = data_augmentation(inputs)               # apply augmentation
x = tf.keras.applications.efficientnet.preprocess_input(x)  # preprocess as EfficientNet expects
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(num_classes, activation='softmax')(x)
model = models.Model(inputs, outputs)

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
Model: "functional_3"
```

Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 224, 224, 3)	0
sequential_1 (Sequential)	(None, 224, 224, 3)	0
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_1 (Dense)	(None, 3)	3,843

```
Total params: 4,053,414 (15.46 MB)
Trainable params: 3,843 (15.01 KB)
```

Task 2_5

This code trains the model using the training dataset `train_ds` to check the performance and validation of the dataset. `EPOCHS = 10` shows how many times the model will go through the entire data and `model.fit(...)` is the function that adjusts the model's weights to improve the predictions.

```
# === STEP 5: Train the model ===

EPOCHS = 10

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
```

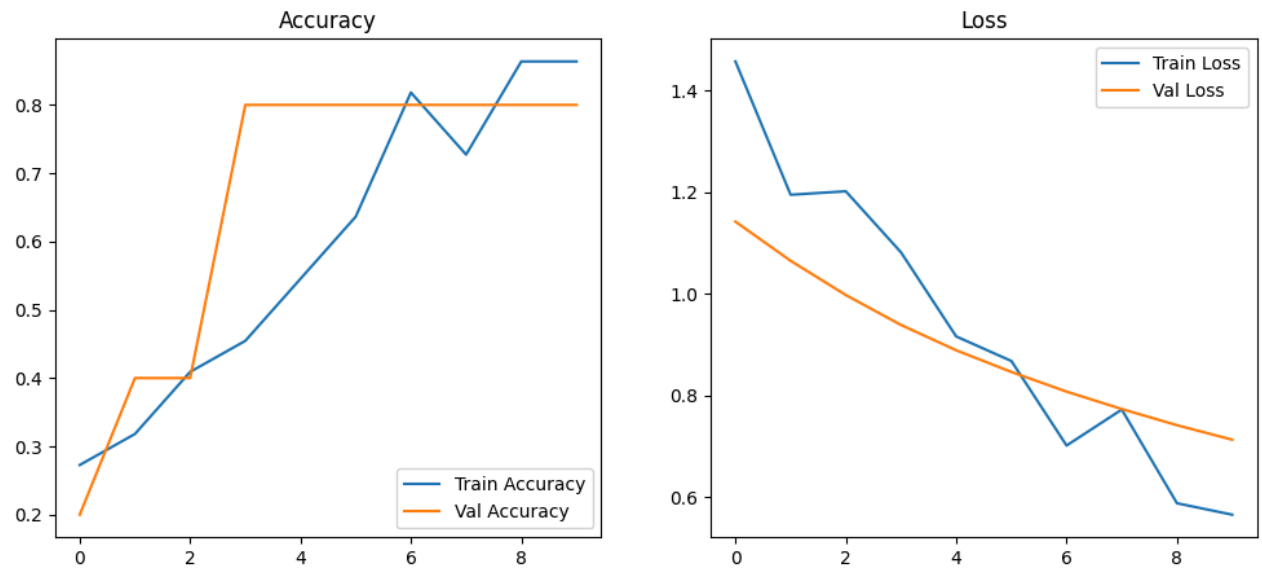
```
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 14s 14s/step - accuracy: 0.2727 - loss: 1.4570 - val_accuracy: 0.2000 - val_loss: 1.1421
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 315ms/step - accuracy: 0.3182 - loss: 1.1949 - val_accuracy: 0.4000 - val_loss: 1.0651
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 318ms/step - accuracy: 0.4091 - loss: 1.2018 - val_accuracy: 0.4000 - val_loss: 0.9978
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 325ms/step - accuracy: 0.4545 - loss: 1.0817 - val_accuracy: 0.8000 - val_loss: 0.9390
Epoch 5/10
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 471ms/step - accuracy: 0.5455 - loss: 0.9164 - val_accuracy: 0.8000 - val_loss: 0.8892
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 525ms/step - accuracy: 0.6364 - loss: 0.8681 - val_accuracy: 0.8000 - val_loss: 0.8465
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 523ms/step - accuracy: 0.8182 - loss: 0.7019 - val_accuracy: 0.8000 - val_loss: 0.8079
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 515ms/step - accuracy: 0.7273 - loss: 0.7726 - val_accuracy: 0.8000 - val_loss: 0.7737
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 522ms/step - accuracy: 0.8636 - loss: 0.5884 - val_accuracy: 0.8000 - val_loss: 0.7419
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 483ms/step - accuracy: 0.8636 - loss: 0.5660 - val_accuracy: 0.8000 - val_loss: 0.7134
```

Task 2_6

This code plots training curves so that I can visually see how it learns. `history.history` stores the accuracy and loss values including (`accuracy, loss)` and validation values `(val_accuracy, val_loss)`. `plt.figure(figsize=(12, 5))` generates two graphs that are side by side. The first graph shows training accuracy and validation accuracy. The second graph shows training loss and validation loss. Finally, `plt.show()` displays both graphs.

```
# === STEP 6: Plot training curves ===

acc, val_acc = history.history['accuracy'], history.history['val_accuracy']
loss, val_loss = history.history['loss'], history.history['val_loss']

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(acc, label='Train Accuracy')
plt.plot(val_acc, label='Val Accuracy')
plt.legend()
plt.title('Accuracy')

plt.subplot(1, 2, 2)
plt.plot(loss, label='Train Loss')
plt.plot(val_loss, label='Val Loss')
plt.legend()
plt.title('Loss')

plt.show()
```



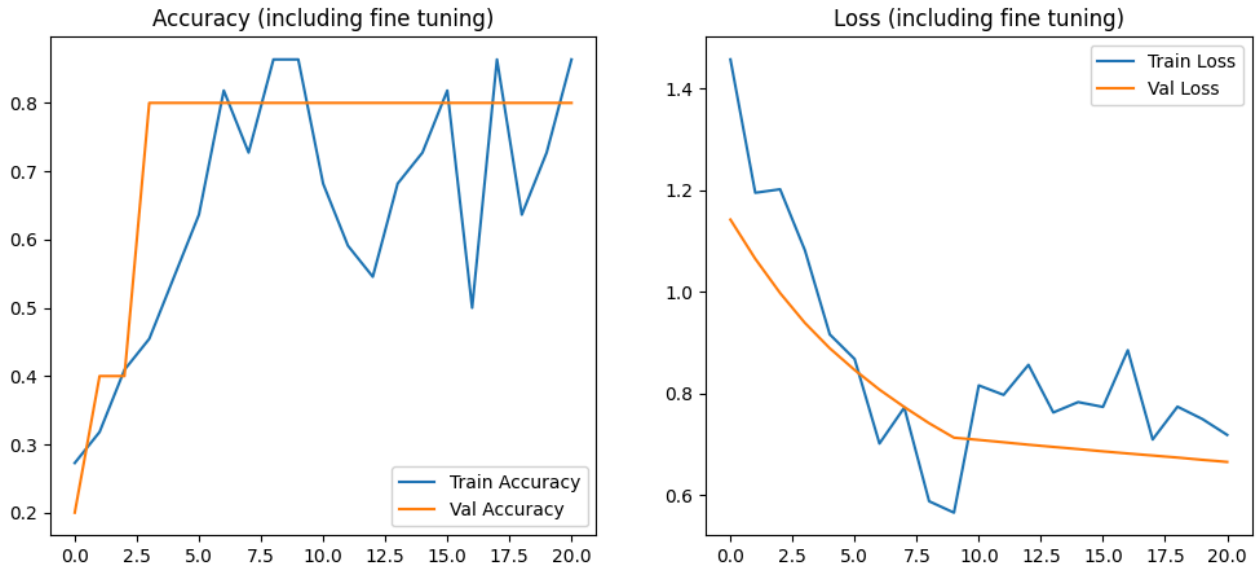
Task 2_7

This code fine tunes the base model form the above code pre-trained from the EfficientNetB0 model. The code `base_model.trainable = True` allows the base model to be trained, the model is then compiled again using a very small learning rate (`Adam(1e-5))` to reduce the risk of damaging the information. `model.fit()` continues on what the previous dataset left off and the new results will be stored in `history_fine` which combines the previous training results so that the final graphs show both the original training and fine tuning.

```
# === STEP 7 (Optional): Fine-tune base model ===

base_model.trainable = True
```

```
# Freeze initial layers, train last layers of EfficientNet
fine_tune_at = 100  # Freeze all layers before this
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

fine_tune_epochs = 10
total_epochs = EPOCHS + fine_tune_epochs

history_fine = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=total_epochs,
    initial_epoch=history.epoch[-1]
)

# Plot fine-tuning results (optional)
acc += history_fine.history['accuracy']
val_acc += history_fine.history['val_accuracy']
loss += history_fine.history['loss']
val_loss += history_fine.history['val_loss']

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(acc, label='Train Accuracy')
plt.plot(val_acc, label='Val Accuracy')
plt.legend()
plt.title('Accuracy (including fine tuning)')

plt.subplot(1, 2, 2)
plt.plot(loss, label='Train Loss')
plt.plot(val_loss, label='Val Loss')
plt.legend()
plt.title('Loss (including fine tuning)')

plt.show()
```

```
Epoch 10/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 32s 32s/step - accuracy: 0.6818 - loss: 0.8162 - val_accuracy: 0.8000 - val_loss: 0.7090
Epoch 11/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 413ms/step - accuracy: 0.5909 - loss: 0.7976 - val_accuracy: 0.8000 - val_loss: 0.7044
Epoch 12/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 594ms/step - accuracy: 0.5455 - loss: 0.8566 - val_accuracy: 0.8000 - val_loss: 0.6996
Epoch 13/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 376ms/step - accuracy: 0.6818 - loss: 0.7630 - val_accuracy: 0.8000 - val_loss: 0.6953
Epoch 14/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 385ms/step - accuracy: 0.7273 - loss: 0.7834 - val_accuracy: 0.8000 - val_loss: 0.6910
Epoch 15/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 385ms/step - accuracy: 0.8182 - loss: 0.7739 - val_accuracy: 0.8000 - val_loss: 0.6865
Epoch 16/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 371ms/step - accuracy: 0.5000 - loss: 0.8854 - val_accuracy: 0.8000 - val_loss: 0.6824
Epoch 17/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 634ms/step - accuracy: 0.8636 - loss: 0.7099 - val_accuracy: 0.8000 - val_loss: 0.6783
Epoch 18/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 372ms/step - accuracy: 0.6364 - loss: 0.7744 - val_accuracy: 0.8000 - val_loss: 0.6743
Epoch 19/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 463ms/step - accuracy: 0.7273 - loss: 0.7501 - val_accuracy: 0.8000 - val_loss: 0.6698
Epoch 20/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 528ms/step - accuracy: 0.8636 - loss: 0.7189 - val_accuracy: 0.8000 - val_loss: 0.6657
```



Task 2_8

This code is used to make a prediction on the selected insect image that is in use. The function `load_and_preprocess_image(img_path)` opens the image from the provided file path and resizes the selected image with `(IMG_SIZE)` which is then converted to a NumPy array. The following code `predict_image(img_path, model, class_names)` then uses that processed image to make a prediction with `model.predict()`. Finnaly, `plt.show()` displays the image.

```
# === STEP 8: Prediction on new images ===

from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

def load_and_preprocess_image(img_path):
    img = image.load_img(img_path, target_size=IMG_SIZE)
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = tf.keras.applications.efficientnet.preprocess_input(img_array)
    return img_array

def predict_image(img_path, model, class_names):
    img_array = load_and_preprocess_image(img_path)
    preds = model.predict(img_array)
    pred_idx = np.argmax(preds[0])
    confidence = preds[0][pred_idx]
    return class_names[pred_idx], confidence

# Example usage:
test_img_path = '/content/drive/MyDrive/Colab Notebooks/Software Technology 1 G (8995)/Week 7/Assets/insects_dataset/test_
predicted_class, confidence = predict_image(test_img_path, model, class_names)

plt.imshow(image.load_img(test_img_path))
plt.title(f'Predicted: {predicted_class} ({confidence * 100:.2f}%)')
```

```
plt.axis('off')
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
```

Predicted: Limnius sp (79.70%)

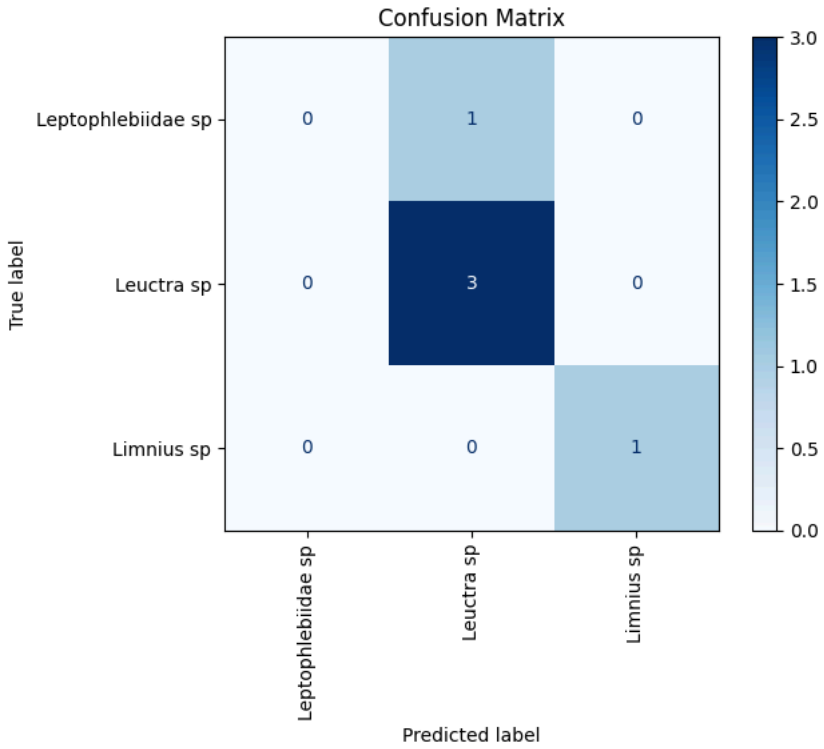
Task 2_9

This code evaluates how well the trained model performs on the validation dataset. To start the code, I imported NumPy, Matplotlib, TensorFlow and tools from `sklearn` for measuring the model's performance. `val_ds` used loops for all the labels and images, then `model.predict(images)` to get prediction probabilities, labels will be stored in `y_true`, and the prediction scores are stored in `y_score`. NumPy creates the Matrix to show each class, then it calculates the weighted precision, recall and F1-scores. Finally, `plt.show()` displays the results.

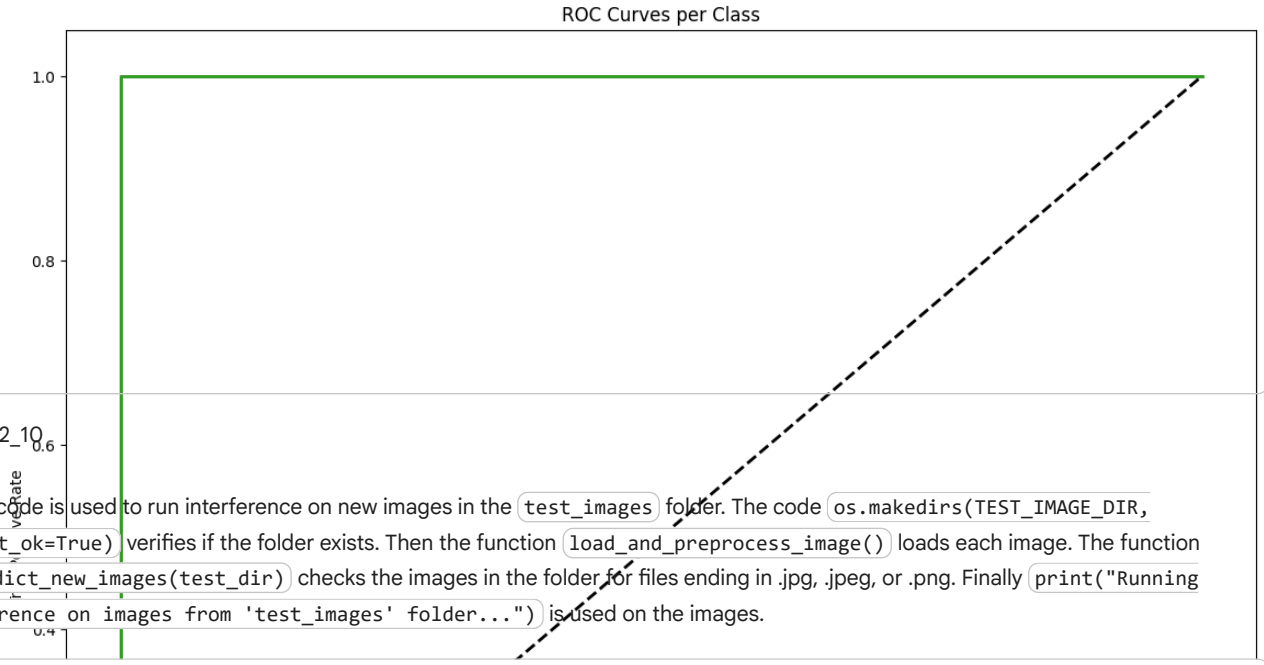
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    precision_recall_fscore_support,
    roc_curve, auc
)
from sklearn.preprocessing import label_binarize
import tensorflow as tf

# Aggregate true labels and predictions from validation dataset
y_true = []
y_pred = []
y_score = []

for images, labels in val_ds:
    preds = model.predict(images)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))
    y_score.extend(preds)

y_true = np.array(y_true)
y_pred = np.array(y_pred)
y_score = np.array(y_score)

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
plt.figure(figsize=(12, 12))
disp.plot(cmap=plt.cm.Blues, xticks_rotation='vertical')
plt.title('Confusion Matrix')
plt.show()

# Precision, Recall and F1-score
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true, y_pred, average='weighted'  # options: 'micro', 'macro', None...
)
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-score:  {f1:.4f}")

# ROC Curve and AUC for multi-class
n_classes = len(class_names)
y_true_bin = label_binarize(y_true, classes=range(n_classes))

fpr = dict()
tpr = dict()
roc_auc = dict()
```

```
plt.figure(figsize=(14, 12))
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
    plt.plot(fpr[i], tpr[i], lw=2, label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves per Class')
plt.legend(loc='lower right', fontsize='small')
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
<Figure size 1200x1200 with 0 Axes>
```




```
Precision: 0.6500
Recall:    0.8000
F1-score:  0.7143
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



Task 2_10

This code is used to run interference on new images in the `test_images` folder. The code `os.makedirs(TEST_IMAGE_DIR, exist_ok=True)` verifies if the folder exists. Then the function `load_and_preprocess_image()` loads each image. The function `predict_new_images(test_dir)` checks the images in the folder for files ending in .jpg, .jpeg, or .png. Finally `print("Running inference on images from 'test_images' folder...")` is used on the images.

```
# === STEP 12: Inference on new images in 'test_images' folder ===

#TEST_IMAGE_DIR = 'test_images'
TEST_IMAGE_DIR = '/content/drive/MyDrive/2026_Teaching/ST1/Week_7_Tut_Lab_Activities/Assets/insects_dataset/test_data'
os.makedirs(TEST_IMAGE_DIR, exist_ok=True)
```

```
def load_and_preprocess_image(img_path, img_size=IMG_SIZE):
    img = image.load_img(img_path, target_size=img_size)
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)  # batch size 1
    return img_array

def predict_new_images(test_dir):
    img_files = [f for f in os.listdir(test_dir) if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    if not img_files:
        print(f"No images found in '{test_dir}'. Please add some images and rerun.")
        return

    for img_file in img_files:
        path = os.path.join(test_dir, img_file)
        img_arr = load_and_preprocess_image(path)
        preds = model.predict(img_arr)
        pred_idx = np.argmax(preds[0])
        confidence = preds[0][pred_idx]

        print(f"Original Image: {img_file} --> Predicted: {class_names[pred_idx]} ({confidence*100:.2f}%)")
        img = image.load_img(path)
        plt.imshow(img)
        plt.axis('off')
        plt.title(f"Prediction: {class_names[pred_idx]} ({confidence*100:.2f}%)")
        plt.show()

print("Running inference on images from 'test_images' folder...")
predict_new_images(TEST_IMAGE_DIR)
```

```
Running inference on images from 'test_images' folder...
No images found in '/content/drive/MyDrive/2026_Teaching/ST1/Week_7_Tut_Lab_Activities/Assets/insects_dataset/test_data'. Pl
```

3. Vision Transformer Model

```
!pip install tensorflow tensorflow-hub tensorflow-datasets matplotlib
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: tensorflow-hub in /usr/local/lib/python3.12/dist-packages (0.16.1)
Requirement already satisfied: tensorflow-datasets in /usr/local/lib/python3.12/dist-packages (4.9.9)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensor
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/lo
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.80.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4
Requirement already satisfied: tf-keras>=2.14.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow-hub) (2.19.0)
Requirement already satisfied: array_record>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (
Requirement already satisfied: dm-tree in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (0.1.10)
Requirement already satisfied: etils>=1.9.1 in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etree
Requirement already satisfied: immutabledict in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (4.3.1)
Requirement already satisfied: promise in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (2.3)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (5.9.5)
Requirement already satisfied: pyarrow in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (18.1.0)
Requirement already satisfied: simple_parsing in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (0.1.8
Requirement already satisfied: tensorflow-metadata in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (
Requirement already satisfied: toml in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (0.10.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (4.67.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.po
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tens
```

```
Requirement already satisfied: einops in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etree]>=1.9
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etree]>=1.9
Requirement already satisfied: zipp in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etree]>=1.9.1
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.19.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorfl
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tenso
```

These commands import the required libraries, import `tensorflow as tf` brings in TensorFlow, `import tensorflow_hub as hub` imports TensorFlow Hub, `import tensorflow_datasets as tfds` imports TensorFlow Datasets, `import matplotlib.pyplot as plt` is used for plotting graphs and displaying images, `import numpy as np` provides tools for working with arrays and numerical data efficiently. Finally, `import os` is Python's built-in operating system module.

```
import tensorflow as tf
import tensorflow_hub as hub
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np
import os
```

This code is used to set the user perimeters that will be used thorough this part of the project. `DATASET_DIR` stores the file path to the training dataset folder and this will also be where the codes will load the images. `IMG_SIZE = 224` sets the dimensions that the images will be resized to, `BATCH_SIZE = 32` means the model will process 32 images at a time, `SEED = 123` sets a fixed random value. Finally, `EPOCHS = 10` makes the model to go through the full dataset 10 times.

```
# === User parameters ===
DATASET_DIR = '/content/drive/MyDrive/2026_Teaching/ST1/Week_7_Tut_Lab_Activities/Assets/insects_dataset/train_data/'  # Yo
IMG_SIZE = 224            # ViT input size
BATCH_SIZE = 32
SEED = 123
EPOCHS = 10
```

Task 3_1

This code loads the images from the `DATASET_DIR` with an 80/20 split. The code `tf.keras.preprocessing.image_dataset_from_directory()` reads images from subfolders which are resized with `IMG_SIZE`. The code `subset='training'` creates the first trained dataset and `subset='validation'` creates a validation dataset. The code `class_names = train_ds.class_names` stores the labels, `num_classes = len(class_names)` counts how many classes there are and finally `print(f"Classes: {class_names}")` displays the results.

```
# === Step 1: Load Dataset with 80/20 split ===

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset='training',
    seed=SEED,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset='validation',
    seed=SEED,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)

class_names = train_ds.class_names
num_classes = len(class_names)
print(f"Classes: {class_names}")

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
```

```
Found 27 files belonging to 3 classes.
Using 22 files for training.
Found 27 files belonging to 3 classes.
Using 5 files for validation.
Classes: ['Leptophlebiidae sp', 'Leuctra sp', 'Limnius sp']
```

Task 3_2

This code is for data augmentation which is used to make the trained images more varied so the later model can learn better without problems. The code `tf.keras.Sequential()` is used to group several images. `RandomFlip("horizontal")` flips the images from left to right, `RandomRotation(0.1)` slightly rotates the images and `RandomZoom(0.1)` zooms in or out accordingly.

```
# === Step 2: Data augmentation ===

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
])
```

Task 3_3

This code loads the ViT features extractor from TF Hub so that the code can work. `vit_url` stores the link to the specific ViT model. The following code `hub.KerasLayer(vit_url, trainable=False)` downloads that model and wraps it as a Keras layer called `vit_layer`.

```
# === Step 3: Load ViT Feature Extractor from TF Hub ===

vit_url = "https://tfhub.dev/sayakpaul/vit_b16_fe/1"
vit_layer = hub.KerasLayer(vit_url, trainable=False)
```

Task 3_4

This code is used to prepare the images classification models using the ViT features extractor. First, `tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))` defines the input layer for the images. The selected images are then passed through `data_augmentation` to be modified while they are being trained, after which they are sent through `vit_layer`. Finally, `tf.keras.Model(inputs, outputs)` creates the full model, and `model.compile()` sets it up for training.

```
#====Step 4:
inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

# Data augmentation
x = data_augmentation(inputs)

# The TF Hub ViT module expects pixel values in [0, 255].
# image_dataset_from_directory typically outputs float32 in [0, 255],
# and data_augmentation maintains this. So, no explicit [0,1] scaling is needed here.

# ViT feature extractor
# Wrap the vit_layer call in a Lambda layer to ensure proper symbolic tensor handling
features = tf.keras.layers.Lambda(lambda y: vit_layer(y))(x);

# Classification head
outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(features)

model = tf.keras.Model(inputs, outputs)

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Time 3_5

This code is used to train the model dataset `train_ds` and see how well it performs on the validation dataset `val_ds` which is stored in `EPOCHS`. The code `model.fit()` uses the images to start the process by comparing the predictions and stored labels. It then evaluates the model on the validation dataset which is all saved in `history` which stores the training loss, validation loss and accuracy.

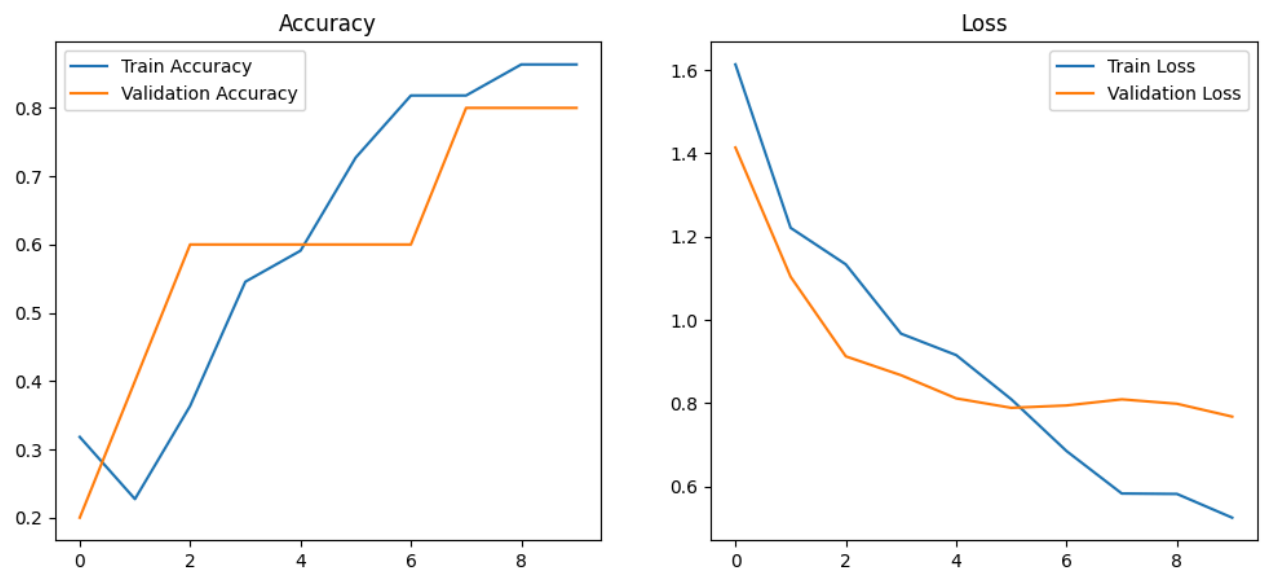
```
# === Step 5: Train the model ===

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
)
```

```
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 21s 21s/step - accuracy: 0.3182 - loss: 1.6131 - val_accuracy: 0.2000 - val_loss: 1.4135
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 685ms/step - accuracy: 0.2273 - loss: 1.2208 - val_accuracy: 0.4000 - val_loss: 1.1033
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 601ms/step - accuracy: 0.3636 - loss: 1.1330 - val_accuracy: 0.6000 - val_loss: 0.9122
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 501ms/step - accuracy: 0.5455 - loss: 0.9667 - val_accuracy: 0.6000 - val_loss: 0.8670
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 488ms/step - accuracy: 0.5909 - loss: 0.9152 - val_accuracy: 0.6000 - val_loss: 0.8113
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 507ms/step - accuracy: 0.7273 - loss: 0.8089 - val_accuracy: 0.6000 - val_loss: 0.7886
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 510ms/step - accuracy: 0.8182 - loss: 0.6847 - val_accuracy: 0.6000 - val_loss: 0.7943
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 527ms/step - accuracy: 0.8182 - loss: 0.5829 - val_accuracy: 0.8000 - val_loss: 0.8089
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 504ms/step - accuracy: 0.8636 - loss: 0.5818 - val_accuracy: 0.8000 - val_loss: 0.7984
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 515ms/step - accuracy: 0.8636 - loss: 0.5248 - val_accuracy: 0.8000 - val_loss: 0.7675
```

Task 3_6

This code is used to plot training and validation accuracy and loss and display them in a graph. The code `plt.figure(figsize=(12, 5))` makes a figure large enough to place two graphs side by side. The first graph shows the training accuracy and validation accuracy from `history.history` which shows the displayed information. The second graph shows the training loss and validation loss. The codes display the `label`, `legend()`, and `title()` so that the graphs are easier to read. Finally, `plt.show()` displays the graphs.

```
# === Step 6: Plot training & validation accuracy/loss ===

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Accuracy')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss')

plt.show()
```



Task 3_7

This code is for the Prediction helper function which makes it easier to use new images in the training model. The code `load_and_preprocess_image(img_path)` loads the image to resize it with `IMG_SIZE, IMG_SIZE` so that it matches the input size so that it can be converted into an array so that it can be scaled by the pixel values 255.0. The second function `predict_image(img_path, model, class_names)` uses the processed image to make a prediction with `model.predict()`.

```
# === Step 7: Prediction helper function ===

def load_and_preprocess_image(img_path):
    img = tf.keras.preprocessing.image.load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
    img_array = tf.keras.preprocessing.image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0
    return img_array

def predict_image(img_path, model, class_names):
    img_array = load_and_preprocess_image(img_path)
    preds = model.predict(img_array)
    pred_idx = np.argmax(preds[0])
    confidence = preds[0][pred_idx]
    return class_names[pred_idx], confidence
```

Task 3_8

This code uses the trained model to display the image. The code `test_img_path` gets the insect image from the file path that is provided to it, then it runs the image through `predict_image(test_img_path, model, class_names)` and when it is done it should have the class name and models score. Next `plt.imshow()` displays the original image, a title for the image is displayed using `plt.title()`. Finally, `plt.show()` displays the image with its prediction on the screen.

```
# === Example prediction ===
test_img_path = '/content/drive/MyDrive/Colab Notebooks/Software Technology 1 G (8995)/Week 7/Assets/insects_dataset/test_c

predicted_class, confidence = predict_image(test_img_path, model, class_names)

plt.imshow(tf.keras.preprocessing.image.load_img(test_img_path))
plt.title(f'Predicted: {predicted_class} ({confidence * 100:.2f}%)')
plt.axis('off')
plt.show()
```

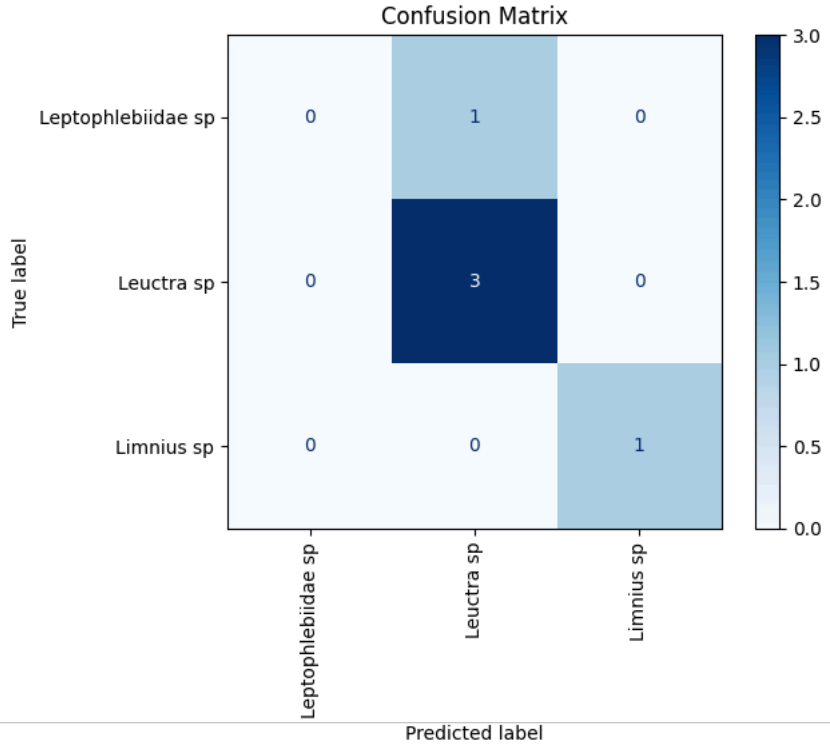
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
```

Taks 3_9

This code uses the trained model's performance on the validation dataset by using several common classification measures. First, it impots the libraries that are required, then `val_ds` is used for loops, then collect the labels using `y_true`, then gather the predicted class labels `y_pred` for the images. This is then converted into a NumPy array to be analysed easily. Then a matrix is created to display how many images from each true class were correctly predicted.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    precision_recall_fscore_support,
    roc_curve, auc
)
from sklearn.preprocessing import label_binarize
import tensorflow as tf

# Aggregate true labels and predictions from validation dataset
y_true = []
y_pred = []
y_score = []

for images, labels in val_ds:
    preds = model.predict(images)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))
    y_score.extend(preds)

y_true = np.array(y_true)
y_pred = np.array(y_pred)
y_score = np.array(y_score)

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
plt.figure(figsize=(12, 12))
disp.plot(cmap=plt.cm.Blues, xticks_rotation='vertical')
plt.title('Confusion Matrix')
plt.show()

# Precision, Recall and F1-score
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true, y_pred, average='weighted'  # options: 'micro', 'macro', None...
)
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-score:  {f1:.4f}")

# ROC Curve and AUC for multi-class
n_classes = len(class_names)
y_true_bin = label_binarize(y_true, classes=range(n_classes))

fpr = dict()
tpr = dict()
roc_auc = dict()

plt.figure(figsize=(14, 12))
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
    plt.plot(fpr[i], tpr[i], lw=2, label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves per Class')
plt.legend(loc='lower right', fontsize='small')
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
<Figure size 1200x1200 with 0 Axes>
```



Task 3.10

```
Precision: 0.6500
Recall:    0.8000
F1-score:  0.7143
/usr/local/lib/python3.12/dist-packages/sklearn/metric
  warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

This code is used to run inference on new images in the `test_images` folder. It would use the `load_and_preprocess_image()` to load the image, resizes it, converts it to an array then scales pixel values by dividing by 255.

ROC Curves per Class

```
# === STEP 12: Inference on new images in 'test_images' folder ===

#TEST_IMAGE_DIR = 'test_images'
TEST_IMAGE_DIR = '/content/drive/MyDrive/2026_Teaching/ST1/Week_7_Tut_Lab_Activities/Assets/insects_dataset/test_data/Copy
os.makedirs(TEST_IMAGE_DIR, exist_ok=True)

# Ensure IMG_SIZE is treated as a tuple for this function
_target_img_size = (IMG_SIZE, IMG_SIZE) if not isinstance(IMG_SIZE, tuple) else IMG_SIZE

def load_and_preprocess_image(img_path, img_size=_target_img_size):
    img = image.load_img(img_path, target_size=img_size)
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)  # batch size 1
    return img_array

def predict_new_images(test_dir):
    img_files = [f for f in os.listdir(test_dir) if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    if not img_files:
        print(f"No images found in '{test_dir}'. Please add some images and rerun.")
        return

    for img_file in img_files:
        path = os.path.join(test_dir, img_file)
        img_arr = load_and_preprocess_image(path)
        preds = model.predict(img_arr)
        pred_idx = np.argmax(preds[0])
        confidence = preds[0][pred_idx]

        print(f"Original Image: {img_file} -> Predicted: {class_names[pred_idx]} ({confidence*100: 2f}%)")
```